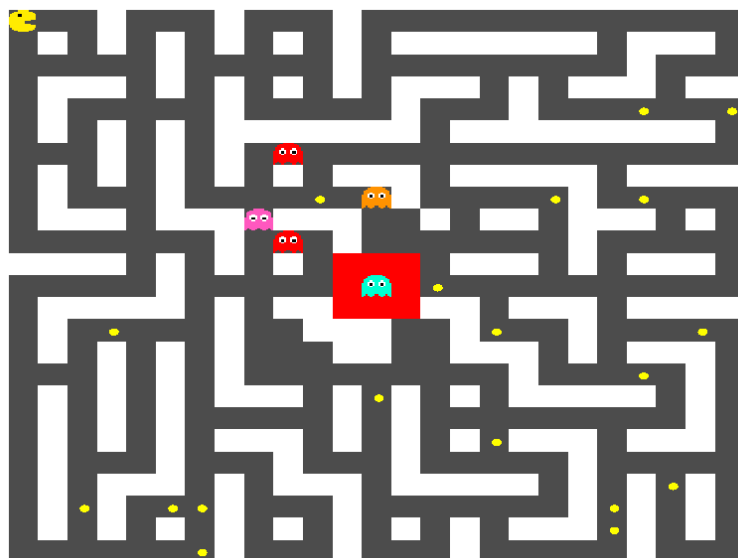


Université Marie & Louis Pasteur
UFR Sciences et Techniques

Projet L3 : Pacman en réseau

Rapport de Projet

Licence CMI Informatique - 3ème année
2025-2026



Thomas COUTANT
Benoît LITAMPHA
Auriane PETER

Encadrant : Julien BERNARD

Remerciements

Table des matières

Remerciements	1
Introduction	4
1 Présentation du projet	5
1.1 Contexte académique	5
1.2 Présentation générale du jeu Pac-Man	5
1.3 Objectifs généraux du projet	6
2 Définition du sujet et problématique	8
2.1 Problématique du jeu en réseau	8
2.2 Choix techniques	8
2.3 Répartition des rôles	9
3 Architecture et conception	10
3.1 Architecture générale	10
3.2 Structure du serveur	11
3.2.1 Mise en place de la structure	11
3.2.2 Gameplay	13
3.3 Structure du client	19
3.4 Protocoles et échanges réseau	19
4 Bilan et perspectives	20
4.1 Compétences acquises	20
4.2 Résultat final	20
4.3 Améliorations possibles	20
Conclusion	21
Sitographie	22
Résumé	23
Mots-Clés	23
Abstract	23
Key- Words	23

Table des figures

1.1	PacMan : le jeu original	6
2.1	Model de Chaine de responsabilité	9
3.1	Diagrammes de classe de l'architecture réseau	11
3.2	Chaîne de responsabilité côté serveur	12
3.3	Plateau de jeu	15
3.4	Vue des traces	17

Introduction

Chapitre 1

Présentation du projet

1.1 Contexte académique

Ce projet a été réalisé dans le cadre du semestre six de la licence informatique de l'Université Marie & Louis Pasteur. Il s'inscrit dans un enseignement visant à mettre en pratique les connaissances acquises au cours de la formation, notamment à travers la réalisation d'un projet informatique de taille conséquente à plusieurs.

Le travail a été mené en groupe de trois étudiants sur une période s'étendant de la mi-octobre à la fin du mois de mars, sous l'encadrement de Julien Bernard, maître de conférences à l'Université Marie & Louis Pasteur. Le projet porte sur le développement d'un jeu vidéo en réseau, de sa conception jusqu'à son implémentation.

Les objectifs pédagogiques de ce projet sont multiples. Il vise en particulier à consolider les compétences en programmation C++, à approfondir la découverte du développement de jeux vidéo, ainsi qu'à mettre en œuvre une architecture réseau de type client-serveur. Le projet permet également de développer des compétences transversales telles que le travail en équipe et la gestion d'un projet informatique sur une durée prolongée.

1.2 Présentation générale du jeu Pac-Man

Le jeu **Pac-Man**, apparu au début des années 1980, est un jeu d'arcade dans lequel le joueur contrôle un personnage évoluant dans un labyrinthe. L'objectif principal est de parcourir l'ensemble du labyrinthe afin de consommer toutes les **pac-gommes** qui s'y trouvent, tout en évitant d'entrer en contact avec des fantômes. Ces derniers constituent les adversaires du joueur et cherchent à bloquer ou intercepter ses déplacements. Certaines **pac-gommes** spéciales permettent temporairement à **Pac-Man** d'inverser les rôles en devenant capable de manger les fantômes, ajoutant ainsi une dimension stratégique au gameplay.

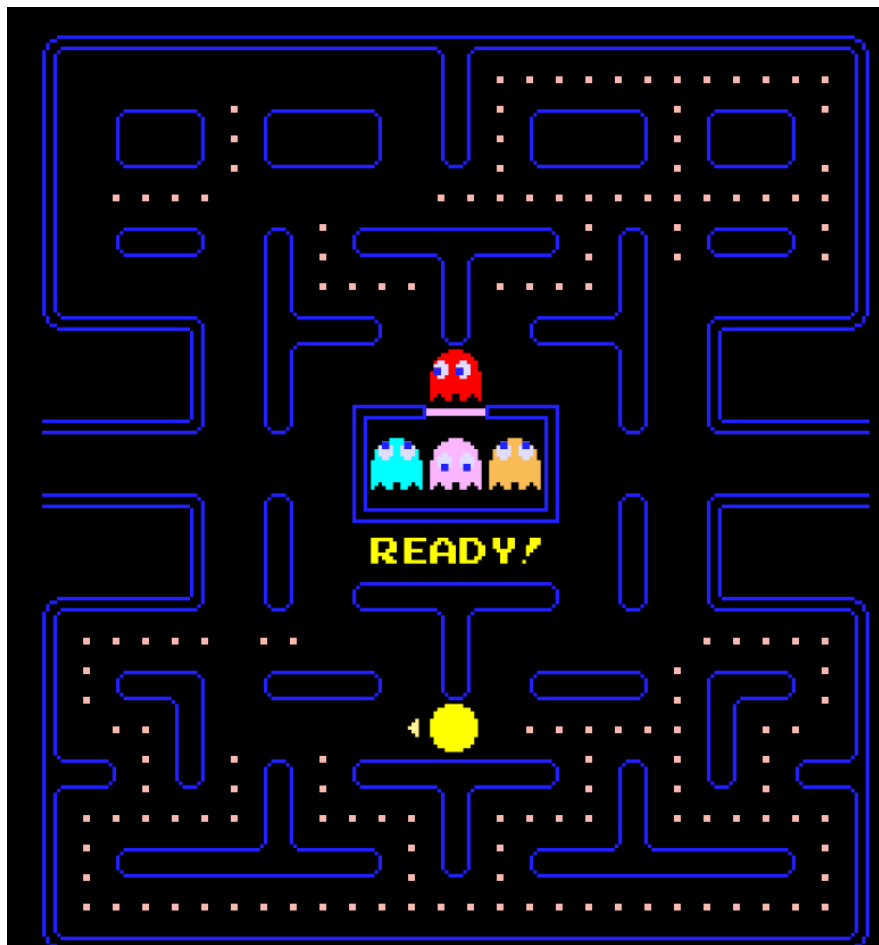


FIGURE 1.1 – PacMan : le jeu original

Dans le cadre de ce projet, le jeu a été adapté afin d'introduire une dimension multijoueur. Une partie oppose un joueur incarnant **Pac-Man** à plusieurs joueurs incarnant des fantômes. Lorsque le nombre de joueurs humains est insuffisant, certains fantômes peuvent être contrôlés par des intelligences artificielles afin de garantir un déroulement cohérent de la partie. Cette approche permet de conserver un équilibre de gameplay, tout en mettant en avant des interactions entre les joueurs.

Le choix de **Pac-Man** comme base pour une adaptation en réseau s'explique par plusieurs raisons. Tout d'abord, ses règles simples et très connues, ce qui permet une prise en main rapide par les joueurs, cela facilite les phases de test et de démonstration. De plus, le gameplay se prête naturellement à une séparation des rôles entre plusieurs joueurs connectés.

1.3 Objectifs généraux du projet

Les objectifs du projet peuvent être divisés en deux catégories principales : les objectifs fonctionnels, liés à l'expérience de jeu, et les objectifs techniques, liés à la conception et à l'implémentation du système.

Du point de vue fonctionnel, l'objectif principal était de concevoir un jeu Pac-Man jouable en réseau, permettant à plusieurs joueurs de participer simultanément à une même partie.

Sur le plan technique, le projet visait à mettre en œuvre une architecture réseau reposant sur un modèle client-serveur. Le serveur devait être capable de gérer les connexions simultanées des joueurs, la création et la gestion des parties, ainsi que l'échange des données nécessaires au bon déroulement du jeu.

Chapitre 2

Définition du sujet et problématique

2.1 Problématique du jeu en réseau

La conception d'un jeu multijoueur en temps réel implique plusieurs contraintes techniques et architecturales. La contrainte majeure concerne la synchronisation des états du jeu entre les différents clients. Chaque joueur doit avoir une vision cohérente de l'ensemble de la partie, incluant les positions de Pac-Man, des fantômes et l'état des pac-gommes. Pour garantir cette cohérence, le projet a opté pour un modèle de serveur autoritaire : toutes les décisions critiques sont validées côté serveur, qui diffuse ensuite les informations mises à jour à tous les clients. Les clients, de leur côté, envoient uniquement leurs intentions de déplacement, qui sont ensuite traitées et synchronisées par le serveur. Les données transitent via des buffers et des queues avant d'être appliquées, assurant un traitement ordonné et fiable des événements du jeu.

2.2 Choix techniques

Le projet a été développé en C++ en utilisant la bibliothèque *Gamedev Framework* (gf), adaptée au développement de jeux vidéo et compatible avec la gestion de l'architecture réseau. La communication réseau repose exclusivement sur le protocole TCP, assurant la fiabilité des échanges entre le serveur et les clients. Les messages échangés sont sérialisés afin de structurer et standardiser les informations transmises.

Le serveur a été conçu selon un modèle multithreadé ou chaîne de responsabilité, reflétant la logique de fonctionnement du jeu : un thread principal gère le serveur et le lobby, tandis que d'autres threads s'occupent des rooms, de la boucle de jeu, des fantômes contrôlés par l'IA et de la communication avec les clients. Cette architecture permet de traiter simultanément les interactions multiples des joueurs et d'assurer une exécution fluide et cohérente des parties.

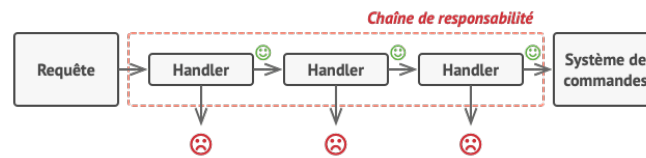


FIGURE 2.1 – Model de Chaîne de responsabilité

2.3 Répartition des rôles

La réalisation de ce projet a été organisée de manière collaborative entre les trois membres du groupe.

Thomas Coutant a principalement travaillé sur le développement du serveur, ainsi que sur la conception de la logique du jeu. Il a également développé un prototype minimal de démarrage, permettant de tester la communication client-serveur avec un simple cube représentant un joueur en mouvement.

Auriane Peter s'est concentrée sur le développement du client, en implémentant l'interface et la gestion des interactions côté joueur.

Benoît Litampha a assuré la communication entre le serveur et le client, participant également à certaines tâches sur le serveur et le client lorsque nécessaire.

Cette répartition a permis une progression simultanée sur plusieurs fronts du projet, tout en garantissant une cohérence globale de l'application.

Chapitre 3

Architecture et conception

3.1 Architecture générale

L'architecture du serveur est conçue pour supporter un jeu multijoueur complet reposant sur une structure modulaire, évolutive et maintenable.

Le serveur s'appuie sur une architecture multithread permettant de gérer plusieurs connexions simultanées sans dégradation des performances. La couche réseau est strictement séparée de la logique de jeu afin d'éviter qu'une opération bloquante — telle qu'une attente de paquet, une latence élevée ou une déconnexion — n'impacte l'exécution globale du système.

Un thread est dédié à la réception des paquets réseau. Les messages reçus sont insérés dans une file d'attente interne, puis traités de manière synchronisée par la logique centrale du serveur. Cette organisation garantit une séparation claire entre la couche communication et la couche métier.

La file de messages agit comme un mécanisme de tampon entre le système réseau et la boucle de jeu à tick fixe. Ainsi, l'exécution des mises à jour périodiques demeure indépendante des variations de latence ou du débit des clients.

Cette architecture renforce la robustesse du système, limite les risques de blocage et permet l'exécution simultanée de plusieurs parties indépendantes au sein du même serveur.

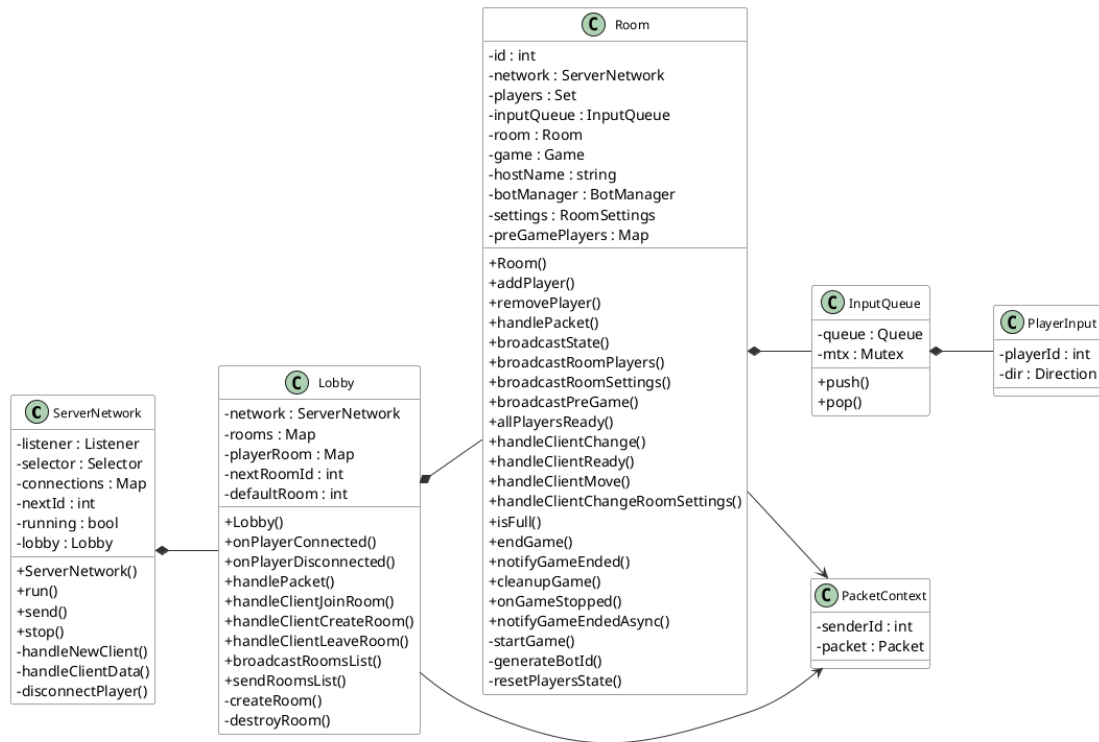


FIGURE 3.1 – Diagrammes de classe de l'architecture réseau

Vue côté serveur - Gestion des packets, des parties, des salles (sans geter et seter)

3.2 Structure du serveur

3.2.1 Mise en place de la structure

Architecture multithread

Le serveur repose sur une architecture multithread conçue pour supporter plusieurs connexions simultanées sans dégradation des performances. La couche réseau est strictement séparée de la logique de jeu afin d'éviter qu'une opération bloquante — attente de paquet, latence élevée ou déconnexion — n'impacte l'exécution globale du système.

Un thread est dédié à la réception des paquets réseau. Les messages entrants sont insérés dans une file d'attente interne, puis traités de manière synchronisée par la logique centrale du serveur. Cette organisation assure une séparation claire entre la couche communication et la couche métier.

La file de messages agit comme un tampon entre le réseau et la boucle de jeu à tick fixe. Ainsi, les mises à jour périodiques du monde restent indépendantes des variations de latence ou du débit des clients.

Cette architecture renforce la robustesse du système et permet l'exécution simultanée de plusieurs parties indépendantes au sein d'un même serveur.

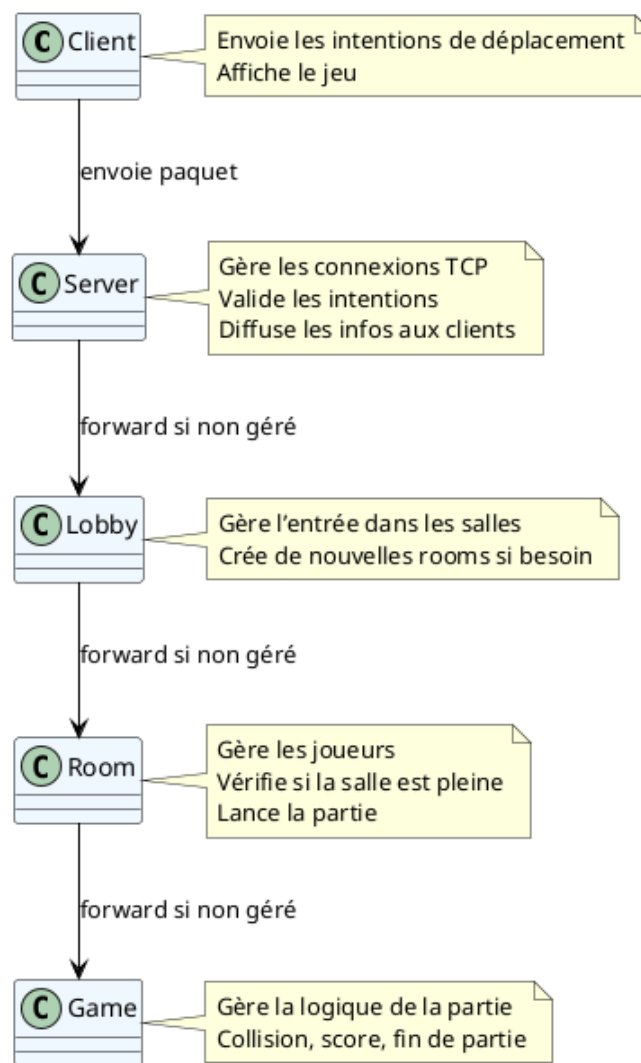


FIGURE 3.2 – Chaîne de responsabilité côté serveur

Architecture modulaire

L'architecture du serveur est organisée en plusieurs couches distinctes possédant chacune une responsabilité clairement définie. Cette structuration modulaire garantit lisibilité, extensibilité et maintenabilité.

La couche *Server* assure la gestion des connexions et le routage des messages. Le *Lobby* centralise l'organisation des joueurs et la création des parties. Chaque partie est encapsulée dans une entité *Room*, responsable de son état propre et de ses joueurs. Enfin, la logique complète du jeu est contenue dans la classe *Game*, indépendante des mécanismes réseau.

L'architecture suit ainsi une chaîne de responsabilité hiérarchique :

Client → Server → Lobby → Room → Game

Chaque couche délègue les traitements spécialisés au niveau inférieur, ce qui permet d'introduire de nouvelles fonctionnalités sans modifier les composants fondamentaux du système.

Structure du plateau

Le plateau est modélisé sous forme d'une grille bidimensionnelle composée de cases murales et de cases traversables. Cette représentation discrète constitue la base logique du système.

Il sert de référence unique pour la détection des collisions, la mise à jour des déplacements, le placement des éléments interactifs ainsi que pour les algorithmes d'intelligence artificielle. La cohérence entre la représentation logique côté serveur et l'affichage côté client garantit une synchronisation fiable du monde de jeu.

Le plateau constitue ainsi une structure centrale sur laquelle reposent la génération procédurale du labyrinthe et les mécanismes décisionnels des fantômes.

3.2.2 Gameplay

Boucle de jeu

La boucle de jeu constitue le cœur du serveur et repose sur un modèle à mise à jour périodique, appelé *tick serveur*. Contrairement à une architecture purement événementielle, l'état du monde est recalculé à intervalle régulier selon une fréquence fixe. Ce fonctionnement garantit un comportement déterministe et centralisé.

À chaque tick, le serveur exécute une séquence ordonnée d'opérations. Les intentions de déplacement reçues des clients sont d'abord traitées, puis les positions des entités sont mises à jour en fonction des règles du jeu. Les collisions sont ensuite évaluées, les états temporaires sont décrémentés, le score est ajusté et les conditions de fin de partie sont vérifiées.

Les rôles sont attribués au début de chaque partie : un joueur incarne Pac-Man tandis que les autres incarnent les fantômes. Chaque rôle possède des contraintes spécifiques influençant les déplacements et les interactions. L'ensemble de ces règles est appliqué exclusivement côté serveur.

Le plateau contient deux types de pac-gommes. Les pac-gommes classiques augmentent le score lors de leur consommation, tandis que les pac-gommes spéciales déclenchent un mode temporaire dit « chasseur ». Ce mode est géré par un système de minuterie interne décrémenté à chaque tick. Lorsque ce compteur atteint zéro, l'état normal est restauré.

Deux systèmes temporels coexistent : un timer global définissant la durée maximale d'une partie, et un timer local associé aux états temporaires. Cette gestion unifiée du temps garantit une cohérence entre les événements et évite toute dépendance au rythme d'exécution des clients.

Les collisions sont évaluées de manière systématique à chaque mise à jour. Un déplacement vers une case murale est annulé. Lors d'une collision entre Pac-Man et un fantôme, l'issue dépend de l'état courant : perte d'une vie en mode normal, ou élimination temporaire du fantôme en mode chasseur. L'intégralité de ces calculs étant effectuée côté serveur, toute tentative de triche ou de désynchronisation est neutralisée.

À la fin de chaque tick, l'état global mis à jour est diffusé à l'ensemble des clients. Ces derniers ne réalisent aucun calcul logique : ils se contentent d'afficher l'état reçu. Cette architecture renforce la cohérence du jeu et s'inscrit dans le modèle de serveur autoritaire défini précédemment.

Plateau et génération procédurale

Le plateau repose sur une génération procédurale de labyrinthe fondée sur une variante de l'algorithme de Prim. Ce choix permet de produire automatiquement des cartes connexes, tout en garantissant une forte variabilité entre les parties.

La génération débute par une initialisation complète du plateau en murs. Une case de départ est ensuite sélectionnée et marquée comme traversable. Ses murs adjacents sont ajoutés à une liste dite *frontier*. À chaque itération, un mur est choisi aléatoirement dans cette liste. S'il sépare une case déjà accessible d'une case encore murale, il est supprimé, la nouvelle case devient traversable, et ses propres murs sont ajoutés à la *frontier*. Le processus se poursuit jusqu'à épuisement de cette liste.

Ce mécanisme produit un arbre couvrant garantissant un labyrinthe connexe sans zones isolées.

Afin d'adapter cette structure aux contraintes du gameplay, plusieurs transformations supplémentaires sont appliquées.

Une zone centrale fixe de taille 3×3 , appelée « hut », est réservée avant la génération. Elle est protégée durant l'exécution de l'algorithme afin de préserver sa structure. Une fonction dédiée assure ensuite son raccordement au labyrinthe principal par au moins une ouverture.

Les coins du plateau, destinés aux positions initiales des joueurs, sont également vérifiés après génération. Si l'un d'eux se retrouve isolé, une ouverture contrôlée est créée pour garantir son accessibilité.

L'algorithme de Prim produisant un arbre sans cycles, des ouvertures supplémentaires sont ajoutées de manière aléatoire afin d'ajouter des chemins. Cette étape enrichit la dynamique de poursuite et évite des trajectoires trop prévisibles.

Enfin, certains culs-de-sac sont supprimés en identifiant les cases ne possédant qu'un unique voisin accessible. La suppression partielle de ces impasses améliore la fluidité des déplacements et limite les situations excessivement punitives.

Des portails latéraux sont également intégrés sur les bords du plateau, permettant à une entité de réapparaître du côté opposé après sortie. Ce mécanisme introduit une dimension stratégique supplémentaire et complexifie les interactions entre joueurs.

Le plateau devient ainsi une structure centrale du système : il sert de support aux collisions, au placement des pac-gommes, ainsi qu'aux algorithmes de poursuite des fantômes. La génération procédurale contribue directement à la rejouabilité et à la diversité stratégique des parties.

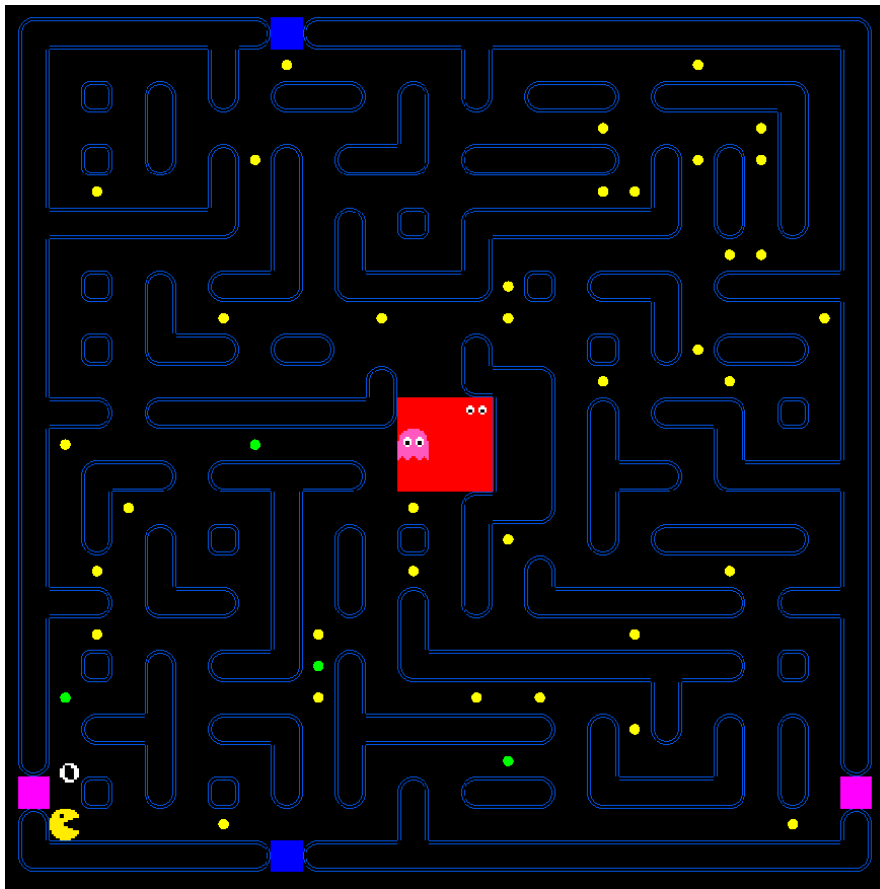


FIGURE 3.3 – Plateau de jeu

Intelligence artificielle des fantômes

L'intelligence artificielle des fantômes repose sur une architecture hybride combinant pathfinding classique, système d'influence local et prise de décision hiérarchique. L'objectif n'est pas uniquement de poursuivre efficacement Pac-Man, mais de produire un comportement crédible, adaptatif et capable de générer des dynamiques collectives.

Architecture décisionnelle hiérarchique À chaque tick serveur, chaque fantôme évalue sa situation selon un ordre de priorité strict.

Lorsque Pac-Man est en mode chasseur, la priorité absolue devient la fuite stratégique vers la cabane centrale. Le plus court chemin est alors calculé à l'aide d'un algorithme de Breadth First Search (BFS), garantissant une trajectoire optimale dans un graphe non pondéré. Ce choix assure robustesse et compatibilité avec les labyrinthes générés procéduralement.

En situation normale, si Pac-Man est détecté dans un rayon de vision défini, le fantôme active un mode de poursuite ciblée. Un BFS à profondeur limitée est exécuté afin de simuler une perception partielle de l'environnement. Cette limitation réduit le coût computationnel tout en conservant un comportement crédible.

En l'absence de menace ou de cible visible, le fantôme adopte un comportement autonome fondé sur un système d'influence distribué.

Système d'influence Chaque case du plateau possède un score dynamique défini par :

$$Score = Attraction - Répulsions$$

Les traces laissées par Pac-Man génèrent une force attractive, tandis que la présence d'autres fantômes induit des forces répulsives différenciées. Le fantôme sélectionne localement la case voisine maximisant ce score selon une approche gloutonne.

Ce mécanisme s'inspire des modèles de colonies de fourmis, où l'information est distribuée dans l'environnement plutôt que centralisée.

Comportement émergent L'interaction entre attraction vers la cible et répulsion inter-agents produit des comportements collectifs sans coordination explicite. Les fantômes tendent naturellement à se disperser, à éviter les regroupements inutiles et à converger progressivement vers Pac-Man en exploitant différentes trajectoires.

Cette émergence résulte uniquement de règles locales simples, démontrant l'efficacité d'un modèle distribué appliqué à un système multi-agents.

Choix algorithmiques L'implémentation combine plusieurs techniques complémentaires : le BFS pour le calcul de chemins optimaux, une version limitée pour la simulation de vision, une recherche gloutonne locale pour l'exploration autonome, ainsi que des heuristiques légères telles que l'utilisation de la distance euclidienne au carré. Un bruit aléatoire contrôlé est également introduit afin d'éviter les cycles déterministes et les oscillations.

Cette approche hybride offre un compromis entre précision stratégique, coût computationnel maîtrisé et richesse comportementale.

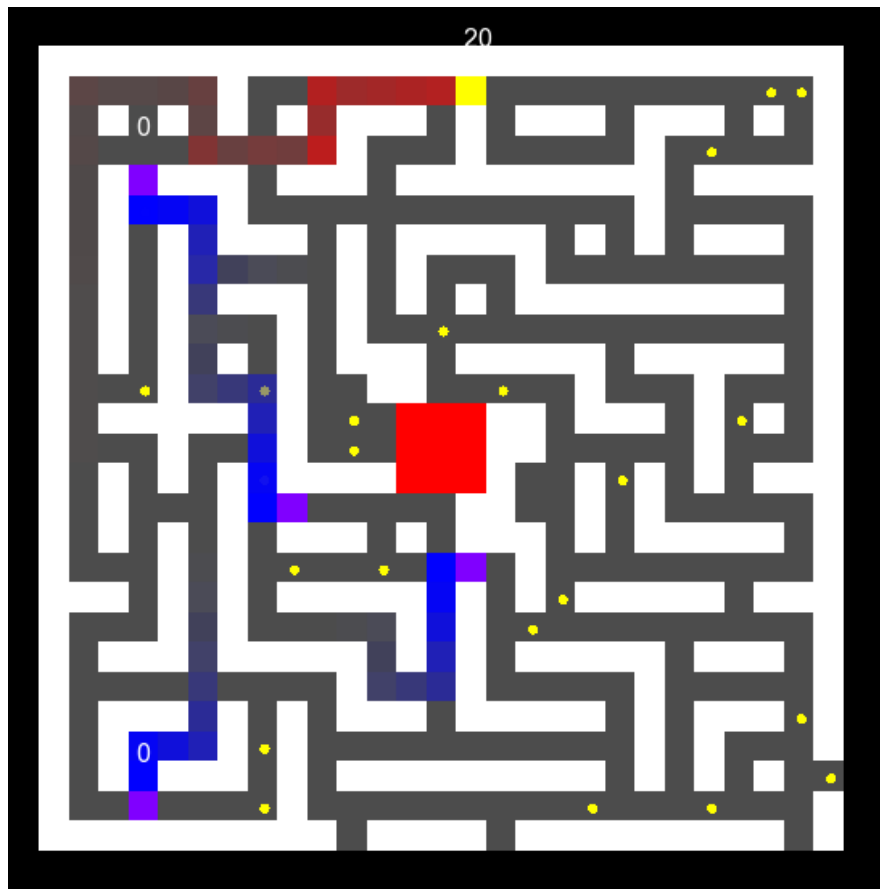


FIGURE 3.4 – Vue des traces

Rouge pour PacMan et Bleu pour les fantômes

Rooms : gestion des parties

Afin de supporter plusieurs parties simultanées, le serveur repose sur un système de *rooms*. Une room représente une instance autonome de jeu possédant son propre état, ses propres joueurs et son propre cycle d'exécution. Cette abstraction permet d'isoler complètement les parties les unes des autres tout en conservant une architecture serveur centralisée.

Chaque room maintient la liste des joueurs connectés, l'attribution des rôles (Pac-Man ou fantômes), les paramètres de la partie ainsi que son état courant (en attente, en cours, terminée). Elle encapsule également sa propre boucle de jeu et son plateau, garantissant qu'aucune donnée n'est partagée entre deux parties distinctes.

Création dynamique et attribution des rôles Lorsqu'un joueur rejoint le lobby, il est dirigé vers une room existante disposant de places libres. Si aucune room n'est disponible, une nouvelle instance est créée dynamiquement. Cette création à la demande permet au serveur d'adapter automatiquement sa structure au nombre de joueurs connectés.

L'attribution des rôles est effectuée au moment du lancement de la partie selon une règle simple : un joueur incarne Pac-Man tandis que les autres deviennent des fantômes.

Ce mécanisme est géré exclusivement côté serveur afin d'éviter toute incohérence ou manipulation côté client.

Gestion et diffusion de l'état Chaque room possède un état centralisé comprenant les positions des joueurs, l'état des pac-gommes et les différents systèmes temporels. À chaque tick, cet état est mis à jour puis diffusé uniquement aux joueurs appartenant à la room concernée.

Ce cloisonnement garantit la cohérence interne de chaque partie et empêche toute interférence entre sessions distinctes. Il constitue un élément clé de la scalabilité du système.

Paramétrisation des parties Les rooms permettent également d'ajuster dynamiquement certains paramètres, tels que la durée maximale d'une partie, le nombre de vies de Pac-Man ou encore la quantité de pac-gommes spéciales. Cette capacité de configuration facilite les phases de test, l'équilibrage du gameplay et l'organisation de parties compétitives aux règles différenciées.

L'introduction du système de rooms marque ainsi une transition vers une architecture modulaire capable de gérer un nombre variable de parties simultanées tout en conservant un modèle serveur autoritaire cohérent.

Règles du jeu

Les règles du jeu ont été définies afin d'instaurer un équilibre entre tension, stratégie et asymétrie des rôles. Chaque camp dispose d'objectifs distincts et de contraintes spécifiques, créant une dynamique d'opposition structurée.

Conditions de victoire Pac-Man remporte la partie s'il parvient à consommer l'ensemble des pac-gommes présentes sur le plateau ou s'il survit jusqu'à l'expiration du timer global. Les fantômes, quant à eux, gagnent en épuisant toutes les vies de Pac-Man avant la fin du temps imparti.

Ce double système de victoire introduit une pression temporelle permanente : Pac-Man doit optimiser ses déplacements, tandis que les fantômes doivent coordonner leurs actions pour limiter ses opportunités.

Pac-gommes spéciales et renversement de dynamique Certaines pac-gommes déclenchent un mode temporaire dit « chasseur ». Durant cette période, Pac-Man devient capable d'éliminer les fantômes au contact. Cet état est strictement limité dans le temps par un compteur décrétement à chaque tick serveur. Les fantômes éliminés réapparaissent après un délai prédéfini, maintenant ainsi l'équilibre global.

Ce mécanisme introduit un renversement ponctuel de dominance, transformant momentanément la proie en prédateur et modifiant profondément les stratégies en cours.

Asymétrie de vision Le système repose également sur une asymétrie informationnelle. Pac-Man dispose d'une vision globale du plateau, tandis que les fantômes ne perçoivent qu'un environnement local restreint.

Cette différence crée une dynamique stratégique spécifique : Pac-Man peut planifier ses trajectoires de manière optimale, alors que les fantômes doivent s'appuyer sur leur

intelligence artificielle distribuée et sur leurs comportements émergents pour localiser efficacement leur cible.

Cohérence du système La combinaison de ces règles avec la génération procédurale du plateau et l'architecture hybride de l'intelligence artificielle produit un gameplay varié et imprévisible. Les interactions entre pression temporelle, asymétrie de vision et renversement temporaire des rôles favorisent l'émergence de stratégies multiples et renforcent la rejouabilité du jeu.

3.3 Structure du client

Affichage, gestion des entrées utilisateur, réception et traitement des messages réseau.

3.4 Protocoles et échanges réseau

Description des trames, types de messages, gestion des événements (connexion, mouvement, fin de partie).

Utilisation de `gf : :TCPSocket`, permettant de faire une transmission de données fiable Socket de connexion + communication coté serveur, communication uniquement pour le client Un thread coté client et serveur pour empiler les packet reçu, puis dépilement dans la boucle de jeu pour être traité

Structures communes utilisé par le client et serveur pour la communication (`BoardCommonn`, `PlayerData`, `RoomSettings`, etc...)

Sérialisation/Désérialisation avec `gf : :Packet.is()` et `gf : :Packet.as()` et la surcharge de `|` avec la classe `Archive`. Déjà implémenté pour les types de base par `gf`

Les structures ont des `gf : :Id`, pour pouvoir les identifier lors de la désérialisation ; Description des strucutres Exemple avec board : le serveur convertit le plateau en `BoardCommon`, puis le sérialise avec `gf : :Packet.is()`, avant de l'envoyer au client. Le client traite le packet en regardant si l'id correspond à la structure `BoardCommon`, puis désérialise avec `gf : :Packet.as()`, avant de traiter les données

Pour chaque modification on retransmet toutes les données

Chapitre 4

Bilan et perspectives

4.1 Compétences acquises

Programmation réseau, architecture logicielle, travail de conception.

4.2 Résultat final

Fonctionnalités implémentées, objectifs atteints ou partiellement atteints.

4.3 Améliorations possibles

idee.txt

Conclusion

Ce projet a permis de mettre en pratique les connaissances acquises durant la formation en informatique, en particulier dans le domaine du développement réseau et de la conception logicielle.

Sitographie

Résumé

Mots-clés

C++ - PacMan - GF - Réseau - [AUTRE]

Abstract

Key-Words

MathLive - JavaScript - TypeScript - HTML - Drag & Drop